Lights Off in Assembly

by Kevin Michael (c00313609)

Lights Off is a game that is played on a grid. Each tile in the grid has an 'On' state or an 'Off' state. Triggering a tile will swap the state of itself, and the state of the tiles that are North, South, East, and West of it. The goal of the game is to turn off all the lights (and save the environment).

General Information:

Level of Complexity: High due to features such as randomizer, 2-dimensional array for grid, seamless user experience and elevated UI.

Number of Addressing Modes: 2 – Immediate and Indirect.

Subroutines Used: RANDOM_DIGIT – Returns random digit into the stack; ASSIGN_GRID – Takes address of specific grid as parameter from stack and assigns it. TAKE_INPUT and more – reusable code even for other projects.

Reusability and Modularity: High – adding in new grids (levels) is very simple. Improving UI will not affect gameplay, Code chunks like the TAKE_INPUT subroutine can be re-used in other projects.

Updated Rubric:

Criterion	Game or Application Difficultly of project	Implementation of core project	C68k Instruction set complexity	Code Structure, Modularity & Error Handling	Documentation & Explanation
Description	Determines if the project is game based or application based and the level of difficultly associated with it	Captures the level that the project has been implemented	The different types of assembly instuctions used and assembly features included (sub-routines, compare, jump, branch, addressing, directives, etc.)	Separation between logic and User IO. Handling of invalid inputs, logic management and workflow	Code comments, design report, or README.
Weight (%)	15	20	30	10	25
Poor (<30%)	Application based - trival problem solving	Fails to execute	basic opcodes, no features	Poorly structured, tightly coupled. No error handling	No documentation.
Satisfactory (30–50%)	Game based - trival problem solving	Partially working; some inconsistencies	More opcode spread, basic features used	Limited structure but readable. Minimal validation	Minimal comments or notes.
Good (50–80%)	Application based - more complex problem solving required	Mostly operational; minor issues	Big range of opcodes, some features used	Mostly modular, minor mixing. Some handling	Some documentation of design.
Excellent (80–100%)	Game based - more complex problem solving required, multi player	Seamless play; fully operational game/application states	Good spread of opcodes, most features implemented	Clean modular code, reusable components. Graceful recovery from errors	Clear explanation of design, architecture, 68k features used and where, application usage, instructions and workflow diagrams.

Design and Architecture:

https://github.com/KevinMichael4441/LightsOff_Assembly

The process of design is captured in the repository above. The game follows the common architecture of initializing values in the beginning, then entering a while-loop that deals with draw and update for each frame.

```
WHILE_LOOP:
    ; Is loop condition true?
    TST.B   D7
    BEQ     ENDOFPROG


    ; Sub-Routine to Display on screen
    BSR     DISPLAY

    ; Sub-Routine to Update Game
    BSR     UPDATE


    ; Run it again
    BRA     WHILE_LOOP
```

This way, the logic for display and update aren't intertwined and can be modified to fit whatever changes are needed. For example, if more text needs to be added to the display, nothing need be changed in the UPDATE sub-routine.

```
; Randoomly choosing the grid to play on
; using time in 1/100 seconds since midnight:
MOVE.B      #8, D0
TRAP #15

; mask upper bits for division to happen (it needs exactly W size)
AND.L   #$0000FFFF, D1
DIVU    #10, D1

; MASK quotient (we only need remainder)
AND.L   #$FFFF0000, D1
SWAP    D1
; Now, D1 should have a value 0-9
```

Initialization:

Mainly a grid is randomly chosen from a set of pre-defined grids. The randomizer is made with the help of Task #8 of TRAP #15.

UPDATE sub-routine:

First, a check is done to see if there's any input from the user with the help of Task #7 of TRAP #15. If there isn't, the program returns from the sub-routine. If there is, it checks what that input is. If it is one of the four directional keys, it moves the player's position in that respective direction. If it is the 'flip' button, then the logic for flipping runs. After the flipping, a check is done to see if the game is won. If the game is won, the player is given the option to play again.

The player's position is stored in D6 as a number from 0-24, indicating the location on the grid. Moving left is simply subtracting one from this value,

moving right is adding one. Similarly, traversing South and North is +/- 5. Checks are done to ensure the movement is valid from the grid's perspective; for example, you cannot travel right if you are on the right-most end of the grid like index 4, 9, 14, 19 and 24.

The flipping is done via the BCHG operator. It first flips the tile that the player is on. Then, using the same logic above, traverses to tiles North, South, East and West of that tile and flips them as well. If the flip is done on the edge or corner, appropriate checks are in place to only flip what's necessary.

The game win check is another while-loop that iterates through the 2D grid and checks if any values are 1 (light turned on). If any such value exists, then that means game is not won and the program returns from the sub-routine.

The DISPLAY sub-routine:

This sub-routine is further divided into three sub-routines. First, the title is displayed. Then, the grid is displayed. Finally, the instructions are displayed. Each has a font color which is changed via Task #21 of TRAP #15.

The title and the instructions are simply NULL terminated strings that are defined in memory and displayed via TRAP #15. However, the display of the grid is far from simple. It features two for-loops nested in another for loop.

The outer for-loop iterates through the rows of the grids. Inner for-loops iterate through the columns of the grid. The first inner for-loop is responsible for printing out the status of light at the respective tile – 0 for turned off and 1 for turned on. Once this is done for a whole row, the cursor is moved to the next line, and we enter the second for-loop.

This loop is responsible solely for printing the underline below the tile that the player is currently located in the grid. For all other tiles, a simple space is printed to fill in. Once this is done for the whole row again, we go to the next row and repeat.

68K features used and where:

Addressing and Data Manipulation:

Immediate addressing is used to set registers to hardcoded values, specifically for TRAP #15 purposes. Indirect addressing is implemented to deal with the grid that's stored in memory.

CLR is used to clear data registers before using them rather than moving #0 to them or using them with garbage values. SWAP is used in tandem with division to have remainder be the lower half of data register to then further work with the remainder.

Arithmetic Operations:

All 4 arithmetic operations are used in the program. Addition and subtraction are mainly used for the loops and for traversing the grid. Multiplication is done in the second for-loop of display to obtain index from row and column using the formula index = row * 5 + col.

Division is used in error checking when traversing the grid and for the randomizer. On dividing, the destination has the quotient in the lower half and the remainder in the higher half. The remainder is extracted and is used as the result of the modulo operation which is why it is useful for the randomizer.

Compare, Jump and Branching

From branching in order to initialize the grid in the beginning of the game to branching back to the START when the game is over, compare and branching is prevalent everywhere in the game. It is also a crucial part in having the loops work. Branching is implemented in the following places: assigning the randomly selected grid, all of the while and for-loops in the game, input handling, error checking (checking if movement/flip is valid) and loading appropriate row and column.

Stack, Stack Pointer (SP) and Sub-Routines

Sub-routines are an integral part of the program. The update and the drawing

of the game are dealt with by sub-routines. Furthermore, the drawing subroutine is split into more subroutines for more specific printing of characters and reading player input is a modular subroutine. The return addresses from these subroutines are stored in memory as a stack. This means that SP is ensured to be restored to avoid issues on restarting game.

More importantly, the ASSIGN_GRID is a subroutine that takes a parameter from the stack and RANDOM_DIGIT is a subroutine that returns a value into the stack. These demonstrate the deeper link between stack and subroutine.

User IO, Tasks, and TRAP #15

Task #4 is used as a delaying mechanism for the player to choose an option between closing the game or playing again. (The integer read is not used so this could have been Task #5 as well.)

Task #7 is used to determine if keyboard input is pending. If it is, then we use Task #5 to read the character from the keyboard and do input handling.

Task #8 is used to obtain the time in 1/100 seconds since midnight for the randomizer.

Task #11 is used to clear the screen on game restart. It is also used to position the cursor back at the beginning and re-print everything rather than clear the screen every frame due to a bug that appears when doing the latter.

Task #12, the very first in the program is used to turn off keyboard echo.

Task #13 is used to display NULL terminated strings onto the screen. This is used to display texts like instructions and the game name.

Task #14 is used to display the space and underscore without carriage return and line feed so that it stays in the grid form.

Task #21 is used to change the font colors.

Task #20 is used to display the status of the current tile – 0 for off and 1 for on.

Logical and Bit Operators, Bit Fields and Bit Masks

The AND operator is used to mask upper bits of data register for division to happen (Task #8 outputs time in 1/100 seconds which is a value bigger than a word so division wouldn't be possible without masking).
Additionally, the AND operator is used to extract remainder or quotient from the destination of division.

BSET is used to set any register to #1 as required for Trap #15 tasks or for setting conditions. BSET is much more efficient than MOVE.B #1, Dn.

BTST is used to specifically test if required value in a data register is 4. This is a very specific use but is indeed more efficient than CMPI.B #4, Dn.

BCHG is used to flip the status of a tile. If the tile is turned on, that is represented by 1 and turned off by 0. BCHG switches 0 to 1 and 1 to 0 without further check.

Instructions:

8 to move North on the grid.

4 to move West on the grid.

5 to trigger the flip on the current tile.

6 to move East on the grid.

2 to more South on the grid.

(Make sure to have num lock turned on!)

Press 8,4,6 or 2 to navigate in the grid. Your current location is highlighted by an underscore below that tile.

Press 5 to flip the tile you are on and the tiles adjacent to the one you are on.

Online Reference / Sample of game:

https://en.wikipedia.org/wiki/Lights_Out_(game)

https://dotnet.github.io/dotnet-console-games/Lights%20Out